

entendendo-puppeter

## Anotações Relacionadas ao puppeteer

### 1. O que é o puppeteer?

é uma biblioteca do [node.js](https://nodejs.org/) desenvolvida pelo google que permite acessar o navegador chromium, e fazer infinitas possibilidades

exemplos:

fazer login;

clicar em like;

tirar print de uma página;

### 2. Para quê ele é utilizado?

Ele é um canivete, sendo amplo para n tarefas, sendo elas:

Automação de tarefas;

Testes automatizados;

Web Scraping

### 3. Sintaxe do puppeteer, ensinar ele o que deve ser feito

Esses comandos servem para abrir, configurar e fechar a instância do Chrome/Chromium.

#### Controle do Navegador (Browser)

- **puppeteer.launch([options]):** É o ponto de partida. Abre uma nova instância do navegador.
  - *Exemplo:* **puppeteer.launch({ headless: false })** (abre o navegador visualmente).
- **browser.newPage():** Abre uma nova aba (página) no navegador.
- **browser.pages():** Retorna uma lista com todas as abas abertas.
- **browser.close():** Fecha o navegador e todos os seus processos.

## Navegação (Page)

Comandos para movimentar a aba aberta pelas URLs da web.

- **page.goto(url):** Navega para o site especificado.
  - *Dica:* Use **{ waitUntil: 'networkidle0' }** como opção para garantir que a página terminou de carregar completamente o JavaScript.
- **page.reload():** Recarrega a página atual (F5).
- **page.goBack()** e **page.goForward():** Voltam ou avançam no histórico de navegação.

## Interação com a Página

Comandos que simulam as ações de um usuário real (clicar, digitar, rolar a tela).

- `page.click(selector)`: Clica em um elemento da página (como um botão ou link) baseado no seletor CSS (ex: `button.submit`).
- `page.type(selector, text)`: Digita um texto em um campo de input.
  - *Dica:* Você pode passar `{ delay: 100 }` para simular uma pessoa digitando devagar.
- `page.select(selector, value)`: Seleciona uma opção em um menu de lista suspensa (`<select>`).
- `page.hover(selector)`: Passa o mouse por cima de um elemento (útil para ativar menus *dropdown*).
- `page.focus(selector)`: Foca em um elemento específico.

## Extração de Dados e Espera (Esquema de Scraping)

Essenciais para web scraping e automação robusta, pois evitam que o código tente interagir com algo que ainda não carregou.

- `page.waitForSelector(selector)`: Pausa a execução do código até que o elemento apareça na tela. Evita muitos erros de "elemento não encontrado".
- `page.locator(selector)`: (Introduzido nas versões mais recentes) Uma forma mais moderna e inteligente de interagir e esperar por elementos automaticamente.
- `page.evaluate(pageFunction)`: Executa um código JavaScript *dentro* do navegador. É o comando mais poderoso para extrair textos, classes ou dados do HTML.
- `page.$eval(selector, pageFunction)`: Atalho que encontra um elemento e já roda uma função nele (equivalente ao `document.querySelector`).
- `page.$$eval(selector, pageFunction)`: Encontra *todos* os elementos que combinam com o seletor (equivalente ao `document.querySelectorAll`).

## Captura de Tela e Arquivos

- `page.screenshot([options])`: Tira uma foto da página. Pode ser da tela inteira (`fullPage: true`) ou apenas de um pedaço.

- `page.pdf([options])`: Salva a página atual como um arquivo PDF (só funciona no modo *headless*).
- `page.content()`: Retorna o código HTML completo e atualizado da página.

# Entendendo-regex

Conceito de regex na área da programação:

**Regex** é uma abreviação para **Regular Expressions** (ou **Expressões Regulares**, em português).

De forma bem simples, uma Regex é uma espécie de "**super busca**". Trata-se de uma sequência de caracteres que forma um **padrão** de texto. Esse padrão serve para você encontrar, validar ou substituir palavras, números ou símbolos em um texto de forma extremamente precisa e automatizada.

## Um exemplo prático para entender a diferença

Imagine que você tem um texto enorme e precisa encontrar todos os números de telefone ali dentro.

- **Sem Regex (Busca comum):** Você teria que digitar o número exato, tipo "99 98888-1111". Se você não souber o número de cabeça, a busca comum não serve.
- **Com Regex:** Você cria um padrão que diz: *"Procure por dois números juntos, depois um espaço, depois cinco números, um hífen e mais quatro números"*. O motor da Regex vai varrer o texto e encontrar **todos** os telefones que seguem esse formato, não importa quais sejam os números.

# Entendendo-scraping

## O que é Scraping? (Conceito Geral)

A palavra *scraping* vem do inglês e significa "**raspar**" ou "**arranhar**". No mundo da tecnologia, **Scraping** é o processo genérico de usar um software para extrair ou "raspar" dados de alguma fonte para que eles possam ser salvos e analisados em outro lugar.

Essa extração pode acontecer em qualquer lugar no seu computador:

- Extrair dados de um monte de arquivos PDF locais.
- Puxar tabelas de várias planilhas do Excel automaticamente.
- Coletar logs de um sistema operacional.

## E o que é Web Scraping?

O **Web Scraping** (ou "Raspagem de Dados da Web") é um tipo específico de scraping. É o ato de usar robôs (*scripts* ou *bots*) para **extrair dados especificamente de sites da internet**.

Em vez de você entrar em um site, selecionar o texto com o mouse, dar **Ctrl + C** e depois **Ctrl + V** em uma planilha (o que seria um trabalho manual massante), você escreve um código — usando ferramentas como o **Puppeteer** ou Python — que faz isso milhares de vezes mais rápido.

## Como o Web Scraping funciona na prática?

O processo geralmente segue três passos simples:

1. **A Requisição:** O robô acessa a URL do site (exatamente como você faz quando digita um endereço no navegador).
2. **A Carga:** O robô "baixa" o código HTML (e o JavaScript, se necessário) daquela página.
3. **A Extração:** O robô varre o código, encontra a informação que você quer (usando seletores de CSS ou **Regex**) e limpa o texto para salvar em um arquivo Excel, JSON ou banco de dados.

## Exemplos reais de uso de Web Scraping

- **Comparadores de Preço:** Sites como o *Buscapé* ou *Zoom* usam web scraping a cada minuto para raspar os preços de produtos em lojas como Amazon, Mercado Livre e Magalu, mostrando onde está mais barato.
- **Monitoramento de Imóveis:** Um investidor pode criar um robô para raspar sites de imobiliárias todos os dias em busca de apartamentos que acabaram de baixar de preço em um determinado bairro.
- **Análise de Sentimento (Marketing):** Empresas raspam comentários de redes sociais ou do Google Maps para entender se as pessoas estão falando bem ou mal de um produto.



- **Agregadores de Vagas:** Sites que juntam vagas de emprego de dezenas de portais diferentes em um único lugar.